> **读者**：使用 iLink 进行日常开发的工程师
> **前提**：项目已完成 Bootstrap（`project-context.md` 和入口文件已就位）
> **版本**：v1.3.00

目录

- 1. 三分钟上手
- 2. 写需求定义——你最重要的工作
- 3. 流水线操作详解
- 4. 审核设计——Human-Gate 实操
- 5. 修订 STAGING 文档——ilink-refine 实操
- 6. 回流与熔断——出了问题怎么办
- 7. 维护 project-context.md
- 8. 多 Story 并行开发
- 9. 常见问题排查
- 10. 反模式——千万别这样做
- 11. 认知模式——Domain Knowledge
- 附录 A：命令速查表
- 附录 B：文件地图
- 附录 C：状态流转图

1. 三分钟上手

1.1 最小操作流程

```
# ① 创建 Story（对应你的 Jira 单号）
/ilink-init kcia-1520

# ② 编辑需求定义（这是你唯一需要手写的文件）
#    打开 iLink-doc/kcia-1520/kcia-1520-需求定义.md，填写内容

# ③ 启动 AI 流水线
/ilink-pm kcia-1520          # AI 分析需求，输出业务合同
/ilink-design kcia-1520      # AI 做技术设计，输出文件清单

# ④ 你审核设计（这是你最重要的审核点）
#    阅读 iLink-doc/kcia-1520/kcia-1520-design.master.md
#    重点看 [TASK_ALLOCATION] 和 [DESIGN_DECISIONS]
ilink-approve kcia-1520      # 审核通过，推进到编码

# ⑤ AI 编码 + 审查
/ilink-coder kcia-1520       # AI 写代码
/ilink-qa kcia-1520          # AI 审查代码

# ⑥ 你做最终确认后提交
git add .
git commit -m "kcia-1520: 功能描述"
```

整个过程中你只做三件事：写需求、审设计、最终提交。

1.2 遇到 STAGING 时

如果 AI 在某个阶段输出了 `STAGING`（表示有不确定项），你有四种处理方式：

```
# 方式 A（推荐）：修订对话，逐条确认
/ilink-refine kcia-1520      # AI 列出所有 [待确认] 项，你逐条决策
ilink-approve kcia-1520      # 确认完毕后推进
```

2. 写需求定义——你最重要的工作

2.1 为什么需求定义这么重要

需求定义是整条流水线的**唯一人类输入**。AI 的 PM → Designer → Coder → QA 全部基于你写的需求定义展开。需求定义的质量直接决定最终代码的质量。

一个写了 10 分钟的需求定义，能省你 2 小时的返工。

2.2 需求定义模板

执行 `/ilink-init <story>` 后，会在 `iLink-doc/<story>/` 下生成模板：

```
# <story> — 需求定义

---

## 1. 功能描述
（用 1-3 句话说清楚要做什么）

---

## 2. 功能范围
- In Scope:
  - （明确要做的事）
- Out of Scope:
  - （明确不做的事）

---

## 3. 验收标准
- AC-01: （可验证的条件）
- AC-02: （可验证的条件）

---

## 4. 约束备注
（已知的技术约束、兼容性要求、特殊注意事项）
```

3. 流水线操作详解

3.1 /ilink-init — 创建 Story

```
/ilink-init kcia-1520
```

执行效果：

- 创建 `iLink-doc/kcia-1520/` 目录
- 生成 `kcia-1520-需求定义.md` 模板

执行后你要做的：编辑需求定义，填写完整内容。

3.2 /ilink-pm — AI 需求分析

```
/ilink-pm kcia-1520
```

AI 读取：需求定义.md + project-context.md
AI 输出：`kcia-1520-pm.master.md`（业务合同）

业务合同包含：

- A 层：功能摘要
- B 层：范围契约、硬约束、验收标准契约、假设与风险
- C 层：给 Designer 的调度通知

你通常不需要审核 PM 输出。 Designer 在做设计时会天然审查 PM 的理解是否正确。

4. 审核设计——Human-Gate 实操

这是你在整个流水线中最重要的审核节点。 设计方向对了，后面基本不出大问题；设计方向错了，后面全部返工。

4.1 打开设计文档

```
# 查看设计文档
cat iLink-doc/kcia-1520/kcia-1520-design.master.md
```

4.2 审核 `[TASK_ALLOCATION]`（必看）

这是 Coder 的"白名单"。**Coder 只会修改这里列出的文件。**

```
---

## [TASK_ALLOCATION]

| 任务 | 修改文件 | 修改类型 | 说明 |
|------|---------|---------|------|
| T-01 | .../FsLoginFailStatHandler.java | 新增 | 登录失败统计处理器 |
| T-02 | .../mapper/FsLoginFailStatMapper.xml | 新增 | MyBatis 查询映射 |
| T-03 | .../mapper/FsLoginFailStatMapper.java | 新增 | Mapper 接口 |
| T-04 | server/fs-cia-mng/.../application.yml | 修改 | 注册功能码 |
```

你要检查：

1. **该有的文件都列了吗？**
   - 比如新增接口通常需要：Handler + Mapper + XML + 注册配置
   - 如果少了 Mapper 接口，说明设计可能遗漏了数据层

5. 修订 STAGING 文档——ilink-refine 实操

5.1 什么是 ilink-refine

`ilink-refine` 是 iLink 的**修订对话协议**。当文档处于 STAGING 状态时，你可以通过它与 AI 逐条确认阻塞项，而**不需要重新生成整个文档**。

5.2 什么时候用

场景	是否用 refine
PM 输出 STAGING，有 `[待确认]` 项	✅ 用 refine 逐条确认
Designer 输出 STAGING，你想微调设计决策	✅ 用 refine 讨论修改
QA 输出 STAGING（上游根因），需讨论处理路径	✅ 用 refine 讨论根因
你已经知道答案，想直接推进	❌ 直接 `ilink-approve`
需要大幅调整方向	❌ 修改上游重跑更合适

5.3 操作流程

```
# 1. 执行 refine
/ilink-refine kcia-1520

# 2. AI 会:
#    - 找到当前 STAGING 的文档（优先级: review > design > pm）
#    - 列出所有阻塞项（[待确认]、[UPSTREAM_BLOCKERS] 等）
#    - 逐条向你确认

# 3. 你逐条给出决策:
#    "AR-03 的假设成立，基础平台组确认本季度不升级 KDEncode"
#    → AI 记录为 [已确认 2026-04-14: 基础平台组确认本季度不升级 KDEncode]

# 4. 所有阻塞项处理完后
```

6. 回流与熔断——出了问题怎么办

6.1 正常回流

QA 审查发现代码有问题，判定根因在 Coder 时：

```
QA 输出 Status: FAIL_BACK_TO_CODER
    ↓
你执行: /ilink-coder kcia-1520    ← Coder 读取 QA 的 [FIX_REQUESTS]
    ↓
Coder 修复后，你再执行: /ilink-qa kcia-1520
```

Coder 在回流时只修复 QA 指出的问题，不会做额外修改。

6.2 回流计数与熔断

每次回流，`.retry_count` 文件中的计数 +1。

```
第 1 次回流 → 正常，Coder 修复
第 2 次回流 → 正常，Coder 修复
第 3 次回流 → 触发熔断警告 ⚠️
```

熔断意味着：AI 自动修复了 3 次还没通过 QA。这通常说明：

- 需求或设计本身有问题
- 或者修复方向偏离了

你需要做的：

1. 阅读最新的 `review.master.md`，看 QA 指出的问题

7. 维护 project-context.md

7.1 什么是 project-context.md

这是你项目的**知识库**，位于项目根目录。所有 AI 角色在执行任何任务前都会读它。

它包含：

- 项目概述和技术栈
- 技术约束（Java 版本、框架约束等）
- 模块职责和依赖关系
- 架构原则
- 构建命令
- 特殊说明

7.2 什么时候更新

发生了什么	是否更新	更新哪个章节
新增了一个子模块	✅	§3 模块职责
升级了 Java 版本	✅	§2 技术约束
换了构建命令或 profile	✅	§6 构建与测试
引入了新的框架或中间件	✅	§2 技术约束 + §4 架构原则
修了一个 bug	❌	—
加了几个接口	❌	—

8. 多 Story 并行开发

8.1 Story 隔离

每个 Story 有独立的目录和文档链：

```
iLink-doc/
├── kcia-1520/     ← Story A 的所有文档
│   ├── kcia-1520-需求定义.md
│   ├── kcia-1520-pm.master.md
│   └── ...
└── kcia-1521/     ← Story B 的所有文档，完全隔离
    ├── kcia-1521-需求定义.md
    └── ...
```

文档层面完全隔离，互不影响。

8.2 什么时候可以并行

场景	能否并行	原因
两个 Story 改不同模块的不同文件	✅ 可以	无代码冲突
两个 Story 改同一个模块的不同文件	⚠️ 小心	可能有编译依赖
两个 Story 改同一个文件	❌ 串行	会有 Git 冲突
两个 Story 都改数据库	⚠️ 小心	migration 顺序可能冲突

8.3 并行开发的建议

1. **先看 `[TASK_ALLOCATION]`**：两个 Story 的设计完成后，对比文件清单是否有交集

9. 常见问题排查

9.1 AI 说"找不到需求定义文件"

原因：你执行了 `/ilink-pm kcia-1520` 但没有先执行 `/ilink-init kcia-1520`。

解决：

```
/ilink-init kcia-1520
# 编辑需求定义后再执行
/ilink-pm kcia-1520
```

9.2 Designer 输出了 STAGING 但我觉得设计没问题

`STAGING` 是 Designer 的**默认状态**，所有设计都需要你审核后手动推进。这不是异常，而是 Human-Gate 机制的正常表现。

解决：审核设计后执行 `ilink-approve kcia-1520`。

9.3 PM 输出了 STAGING，有 `[待确认]` 项

解决：

```
# 推荐：使用 refine 逐条确认
/ilink-refine kcia-1520
# AI 会列出所有 [待确认] 项，你逐条给出决策
# 完成后执行 ilink-approve

# 或者：直接推进（承担风险）
ilink-approve kcia-1520
```


9.4 Coder 改了不在设计清单里的文件

10. 反模式——千万别这样做

❌ 反模式 1：需求写一句话就开工

```
---

## 1. 功能描述
优化登录
```

后果：PM 输出的业务合同范围模糊 → Designer 猜测设计方向 → Coder 写的代码可能不是你想要的 → 来回返工 3 轮。

正确做法：花 10 分钟写清楚功能范围和验收标准。

❌ 反模式 2：不审设计就 approve

```
/ilink-design kcia-1520
ilink-approve kcia-1520    # 看都不看就 approve
```

后果：设计方向如果有偏差，Coder 按错误方向写的代码，QA 按错误基线审查，全链路错。Coder 写的代码越多，浪费越大。

正确做法：至少看 `[TASK_ALLOCATION]` 的文件列表和 `[DESIGN_DECISIONS]`，确认方向正确。

❌ 反模式 3：在 Coder 阶段口头追加需求

```
"代码写得不错，顺便把登录日志也改一下吧"
```

后果：Coder 有白名单约束，追加的需求不在设计文件清单里。要么 Coder 拒绝（你觉得不灵活），要么 Coder 违反协议（QA 会标记偏差）。

正确做法：追加的需求开一个新 Story，或回到 Designer 阶段更新设计。

11. 认知模式——Domain Knowledge

11.1 什么是认知模式

iLink v1.3 新增了**认知模式**，与交付模式（PM→Designer→Coder→QA）并行的独立工作线。

交付模式：AI 帮团队完成开发任务（写代码）
认知模式：AI 帮团队沉淀对现有代码的深度理解（写知识文档）

两条工作线独立运行，互不依赖。

11.2 什么是 Domain Knowledge

Domain Knowledge 是 AI 阅读现有源码后生成的结构化认知文档，包含 10 个标准章节：

章节	内容
§1 模块概述	定位、职责边界、技术栈
§2 业务实体	核心数据结构和关键字段
§3 流程概览	核心业务流程（主流程 + 分支）
§4 内部机制	缓存策略、线程模型、状态机等
§5 业务规则	从代码中提取的硬编码规则
§6 设计决策	架构选型和权衡
§7 配置参数	可调参数清单
§8 故障模式	已知的失败路径和恢复策略
§9 实现品评	与国际同类产品对标（如 Keyclock、Redis）

附录 A：命令速查表

核心流水线命令（交付模式）

命令	阶段	谁执行	输出文件
`/ilink-init <story>`	初始化	人类	需求定义模板
`/ilink-pm <story>`	需求分析	AI	`<story>-pm.master.md`
`/ilink-design <story>`	技术设计	AI	`<story>-design.master.md`
`ilink-approve <story>`	审核推进	人类	（更新 Metadata Status）
`/ilink-coder <story>`	编码	AI	源码文件 + `<story>-code.master.md`
`/ilink-qa <story>`	审查	AI	`<story>-review.master.md`

认知模式命令

命令	阶段	谁执行	输出文件
`/ilink-domain <module>`	认知分析	AI	`domain/<module>-domain-knowledge.md`

辅助命令

命令	用途	谁执行
`/ilink-refine <story>`	修订 STAGING 文档（逐条确认阻塞项）	AI + 人类
`ilink-status [story]`	查看流水线状态与下一步建议	人类
`/ilink-bootstrap`	项目冷启动（生成项目知识库）	AI

附录 B：文件地图

```
<project_root>/
├── project-context.md              ← 项目知识库（AI 每次都读）
├── CLAUDE.md                       ← Claude 入口路由
├── AGENTS.md                       ← Codex/其他 AI 入口路由
│
├── iLink/                          ← 框架文件（不要修改）
│   ├── iLink-root-spec.md             ← 根规范
│   ├── iLink-implementation-guide.md  ← 实施手册
│   └── souls/                         ← 角色规范
│       ├── universal.soul.md
│       ├── pm.soul.md
│       ├── design.soul.md
│       ├── coder.soul.md
│       ├── qa.soul.md
│       └── domain.soul.md              ← 认知模式：Domain Engineer
│
├── iLink-doc/                      ← 文档归档（提交到 Git）
│   ├── domain/                        ← Domain Knowledge（认知模式）
│   │   └── <module>-domain-knowledge.md
│   ├── kcia-1520/
│   │   ├── kcia-1520-需求定义.md       ← 你写的
│   │   ├── kcia-1520-pm.master.md      ← AI 写的
│   │   ├── kcia-1520-design.master.md  ← AI 写的，你审核的
│   │   ├── kcia-1520-code.master.md    ← AI 写的
│   │   └── kcia-1520-review.master.md  ← AI 写的
│   └── kcia-1521/
│       └── ...
│
├── .claude/commands/               ← Claude CLI 命令
├── .codex/commands/                ← Codex CLI 命令
├── .qoder/commands/                ← Qoder CLI 命令
└── src/                            ← 源代码
```

提交规则：

- ✅ 提交：`iLink-doc/`、`project-context.md`、`CLAUDE.md`、`AGENTS.md`、`iLink/`、源码
- ❌ 不提交：`.retry_count`（信号文件）

附录 C：状态流转图

```
┌──────────────────┐
│   /ilink-init    │
│ (人类创建Story)  │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│  编辑需求定义    │
│  (人类手写)      │
└──────────────────┘
         │
         ▼
┌──────────────────┐
│   /ilink-pm      │     STAGING
│ (AI 需求分析)    │──────→ /ilink-refine 确认
└──────────────────┘        → ilink-approve 推进
         │
PENDING_DESIGNER
         │
         ▼
┌──────────────────┐
│  /ilink-design   │
│ (AI 技术设计)    │
└──────────────────┘
         │
     STAGING ← 默认
         │
         ▼
⛔ ┌──────────────────┐
   │  人类审核设计    │
   │  /ilink-refine   │ ← 可选: 修订设计
   │  ilink-approve   │ ← 推进
   └──────────────────┘
         │
PENDING_CODER
         │
         ▼
┌──────────────────┐
│  /ilink-coder    │
│  (AI 编码)       │
└──────────────────┘
         │
     PENDING_QA
         │
         ▼
┌──────────────────┐
│   /ilink-qa      │
│ (AI 代码审查)    │
└──────────────────┘
         │
   ┌─────┴─────┬─────┬─────┐
   │           │     │     │
   ▼           ▼     ▼     ▼
COMPLETED  FAIL_BACK_TO  STAGING
 (通过)    _CODER       (上游问题)
   │        (代码问题)     │
   ▼           ▼           ▼
git commit  /ilink-coder  /ilink-refine
            (回流修复)    (讨论根因)
               │
               ▼
            /ilink-qa
            (重新审查)
               │
            ≥3次回流?
            ├─ 否 → 继续
            └─ 是 → ⚠️ 熔断, 人工介入
```